Chuyue（初悦）— AI 人生模拟器 完整技术文档

> ***版本***：*v1.0.0 | * ***日期***：*2026-02-20 | * ***作者***：*悠然若妃 | * ***原创作品，盗版必究***

目录

1. 项目概述
2. 技术栈与依赖
3. 系统架构
4. 目录结构
5. 数据库设计
6. 后端 API 接口
7. 前端状态管理
8. AI 引擎设计
9. 主题系统
10. 用户流程
11. 多剧本系统
12. 云端存档系统
13. 账号认证系统
14. 部署说明

---

1. 项目概述

Chuyue（初悦） 是一款以 AI 为核心驱动力的文字冒险人生模拟器。玩家通过创建角色、选择剧本世界观，与 AI 共同书写专属的命运故事。游戏采用「三选一 + 自由输入」的交互

机制，每个回合 AI 自动生成三个差异化剧情选项，玩家也可以自由输入任意行动，由 AI 实时响应并推进故事。

本项目的核心设计理念是**沉浸感优先**：精心设计的三套国风主题（少女元氣、橘子汽水、高贵黑金）、流畅的粒子动画、细腻的 AI 叙事，共同构建出一个让玩家愿意反复回来的故事世界。

与市面上同类产品相比，Chuyue 的差异化优势体现在以下三个维度：

维度	Chuyue 特色
AI 推动机制	每 4 回合强制触发意外转折，每 7 回合引入新 NPC，避免剧情停滞
多剧本管理	玩家可创建最多 20 个独立剧本，随时切换世界观继续游戏
双重认证	支持 Manus OAuth 与独立游戏账号两套认证体系并行

---

2. 技术栈与依赖

2.1 核心框架

层级	技术	版本	用途
前端框架	React	19.2.1	UI 渲染与组件管理
后端框架	Express	4.21.2	HTTP 服务器
全栈类型安全	tRPC	11.6.0	前后端类型共享 RPC
数据库 ORM	Drizzle ORM	0.44.5	类型安全的数据库操作
数据库	MySQL / TiDB	—	持久化存储
样式框架	Tailwind CSS	4.x	原子化 CSS
UI 组件库	shadcn/ui + Radix UI	—	无障碍 UI 基础组件
状态序列化	superjson	1.13.3	Date 等类型安全传输
数据请求	TanStack Query	5.90.2	服务端状态管理

2.2 AI 与特殊功能

功能	技术/库	说明
AI 对话	Manus Built-in LLM / OpenAI 兼容 API	双模式：内置代理无需配置，外部 API 自定义
Markdown 渲染	streamdown 1.4.0	AI 输出的流式 Markdown 渲染
密码加密	bcryptjs 3.0.3	游戏账号密码哈希
JWT 认证	jose 6.1.0	游戏账号 Session 签发
动画	framer-motion 12.x	页面过渡与微交互
文件存储	AWS S3 SDK 3.x	云端文件存储

2.3 开发工具链

工具	版本	用途
TypeScript	5.9.3	全栈类型检查
Vite	—	前端构建与 HMR
tsx	—	服务端 TypeScript 执行
Vitest	2.1.9	单元测试框架
drizzle-kit	0.31.4	数据库迁移工具
pnpm	—	包管理器

---

3. 系统架构

3.1 整体架构图

```
┌──────────────────────────────────────────────────────────────┐
│                        客户端（浏览器）                        │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  React 19 + Tailwind 4 + shadcn/ui                   │    │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────────┐    │    │
│  │  │GameContext│  │tRPC Client│  │ aiEngine.ts  │    │    │
│  │  │(useReducer)│  │(TanStack) │  │(AI调用层)    │    │    │
│  │  └──────────┘  └──────────┘  └──────────────┘    │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────┬──────────────────────────────────┘
                           │ HTTP / tRPC (JSON-RPC over HTTP)
┌──────────────────────────▼──────────────────────────────────┐
│                      Express 服务器                          │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  tRPC Router                                         │    │
│  │  ├── auth (Manus OAuth)                              │    │
│  │  ├── account (游戏账号注册/登录)                      │    │
│  │  ├── game (存档/AI代理/API测试)                       │    │
│  │  ├── scripts (剧本管理)                               │    │
│  │  └── system (通知)                                    │    │
│  └──────────────────────────────────────────────────────┘    │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  Drizzle ORM → MySQL/TiDB                            │    │
│  │  tables: users / game_saves / game_accounts /        │    │
│  │          game_scripts                                │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
                           │
┌──────────────────────────▼──────────────────────────────────┐
│                        外部服务                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐    │
│  │ Manus OAuth  │  │ Built-in LLM │  │  AWS S3      │    │
│  │ (用户认证)   │  │ (AI代理)     │  │ (文件存储)   │    │
│  └──────────────┘  └──────────────┘  └──────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

3.2 数据流说明

前端所有数据请求均通过 tRPC 客户端发往 `/api/trpc` 端点，服务器根据 procedure 名称路由到对应处理函数。AI 调用有两条路径：当用户开启「内置 AI 代理」时，前端调用 `trpc.game.aiProxy.mutate()`，由服务端使用 `BUILT_IN_FORGE_API_KEY` 调用 LLM；当用户配置了自己的 API Key 时，前端直接发起 `fetch` 请求到指定的 OpenAI 兼容端点。

---

4. 目录结构

```
life-sim/
├── client/                    # 前端代码
│   ├── public/                # 静态资源
│   └── src/
│       ├── _core/hooks/       # 核心 hooks (useAuth)
│       ├── components/
│       │   ├── game/          # 游戏核心组件 (16个)
│       │   │   ├── WelcomePage.tsx    # 欢迎首页
│       │   │   ├── LoginPage.tsx      # 游戏账号登录/注册
│       │   │   ├── ApiConfigPage.tsx  # API 配置页
│       │   │   ├── AppearancePage.tsx # 外观主题设置
│       │   │   ├── SetupPage.tsx      # 角色创建向导
│       │   │   ├── MainGame.tsx       # 主游戏容器
│       │   │   ├── StoryPage.tsx      # 故事/剧情页 (核心)
│       │   │   ├── ScriptPage.tsx     # 剧本管理页
│       │   │   ├── SocialPage.tsx     # NPC 社交页
│       │   │   ├── InventoryPage.tsx  # 背包道具页
│       │   │   ├── NotesPage.tsx      # 记事本页
│       │   │   ├── SettingsPage.tsx   # 游戏设置页
│       │   │   ├── TopBar.tsx         # 顶部状态栏
│       │   │   ├── BottomNav.tsx      # 底部导航栏
│       │   │   ├── ModalManager.tsx   # 弹窗管理器
│       │   │   └── ThemeCanvas.tsx    # 主题粒子画布
│       │   └── ui/            # shadcn/ui 基础组件 (40+个)
│       ├── contexts/
│       │   └── GameContext.tsx        # 全局游戏状态 (核心)
│       ├── hooks/
│       │   └── useSaveGame.ts         # 云端存档 hook
│       ├── lib/
│       │   ├── aiEngine.ts            # AI 调用引擎 (核心)
│       │   ├── i18n.ts                # 多语言支持
│       │   └── trpc.ts                # tRPC 客户端绑定
│       ├── pages/
│       │   └── Home.tsx               # 根页面 (GameProvider 挂载点)
│       └── App.tsx                    # 路由配置
├── server/
│   ├── _core/                 # 框架核心 (不建议修改)
│   │   ├── llm.ts             # LLM 调用封装
│   │   ├── oauth.ts           # Manus OAuth 处理
│   │   ├── context.ts         # tRPC 上下文
│   │   └── ...
│   ├── routers/
```

```
│   │   ├── game.ts            # 游戏存档 + AI 代理路由
│   │   ├── account.ts         # 游戏账号认证路由
│   │   └── scripts.ts         # 剧本管理路由
│   ├── routers.ts             # 主路由聚合
│   ├── db.ts                  # 数据库查询辅助函数
│   └── storage.ts             # S3 文件存储辅助函数
├── drizzle/
│   └── schema.ts              # 数据库表结构定义
├── shared/
│   ├── const.ts               # 前后端共享常量
│   └── types.ts               # 共享类型定义
├── server/scripts.test.ts     # 剧本路由单元测试
├── server/auth.logout.test.ts # 认证路由单元测试
├── todo.md                    # 项目任务追踪
└── package.json
```

---

5. 数据库设计

5.1 表结构总览

表名	用途	主要字段
`users`	Manus OAuth 用户	id, openId, name, email, role
`game_accounts`	游戏独立账号	id, username, passwordHash, salt, displayName
`game_saves`	游戏存档	id, userId, slotName, scriptId, playerData(JSON), round
`game_scripts`	剧本配置	id, userId, name, genre, worldSetting(JSON), isActive

5.2 users 表

```sql
CREATE TABLE users (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  openId      VARCHAR(64) NOT NULL UNIQUE,   -- Manus OAuth 唯一标识
  name        TEXT,
  email       VARCHAR(320),
  loginMethod VARCHAR(64),
  role        ENUM('user','admin') DEFAULT 'user' NOT NULL,
  createdAt   TIMESTAMP DEFAULT NOW() NOT NULL,
  updatedAt   TIMESTAMP DEFAULT NOW() ON UPDATE NOW() NOT NULL,
  lastSignedIn TIMESTAMP DEFAULT NOW() NOT NULL
);
```

5.3 game_accounts 表

```sql
CREATE TABLE game_accounts (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  username      VARCHAR(32) NOT NULL UNIQUE,  -- 2-20字符，字母/数字/中文
  password_hash VARCHAR(128) NOT NULL,         -- bcrypt 哈希
  salt          VARCHAR(32) NOT NULL,          -- 随机盐值
  display_name  VARCHAR(32),                   -- 可选显示名
  createdAt     TIMESTAMP DEFAULT NOW() NOT NULL,
  updatedAt     TIMESTAMP DEFAULT NOW() ON UPDATE NOW() NOT NULL,
  lastSignedIn  TIMESTAMP DEFAULT NOW() NOT NULL
);
```

5.4 game_saves 表

```sql
CREATE TABLE game_saves (
  id              INT AUTO_INCREMENT PRIMARY KEY,
  userId          INT NOT NULL,                  -- 关联 users.id 或 game_accounts.id
  slotName        VARCHAR(64) DEFAULT 'auto',    -- 存档槽名称
  scriptId        INT,                           -- 关联 game_scripts.id
  playerData      TEXT,                          -- 玩家角色 JSON
  npcsData        TEXT,                          -- NPC 列表 JSON
  storyData       TEXT,                          -- 故事历史 JSON（最近50条）
  inventoryData   TEXT,                          -- 背包 JSON
  notesData       TEXT,                          -- 记事本 JSON
  chatData        TEXT,                          -- 聊天记录 JSON
  apiConfigData   TEXT,                          -- API 配置 JSON（不含 key）
  appearanceData  TEXT,                          -- 外观配置 JSON
  scriptData      TEXT,                          -- 剧本设定 JSON
  round           INT DEFAULT 0,
  gameYear        INT DEFAULT 2024,
  gameMonth       INT DEFAULT 1,
  gameDay         INT DEFAULT 1,
  createdAt       TIMESTAMP DEFAULT NOW() NOT NULL,
  updatedAt       TIMESTAMP DEFAULT NOW() ON UPDATE NOW() NOT NULL
);
```

5.5 game_scripts 表

```sql
CREATE TABLE game_scripts (
  id           INT AUTO_INCREMENT PRIMARY KEY,
  userId       INT NOT NULL,
  name         VARCHAR(64) NOT NULL,           -- 剧本名称
  description  TEXT,                           -- 剧本简介
  genre        VARCHAR(32) DEFAULT '现代',     -- 类型标签
  worldSetting TEXT,                           -- 世界观设定 JSON
  coverColor   VARCHAR(16) DEFAULT '#CBAF98',  -- 封面主色
  coverIcon    VARCHAR(8) DEFAULT '📖',        -- 封面图标
  isActive     BOOLEAN DEFAULT FALSE,          -- 是否为当前激活剧本
  totalRounds  INT DEFAULT 0,                  -- 累计游玩回合数
  lastPlayedAt TIMESTAMP DEFAULT NOW(),
  createdAt    TIMESTAMP DEFAULT NOW() NOT NULL,
  updatedAt    TIMESTAMP DEFAULT NOW() ON UPDATE NOW() NOT NULL
);
```

5.6 worldSetting JSON 结构

`game_scripts.worldSetting` 字段存储剧本世界观的 JSON 对象，结构如下：

```json
{
  "era": "现代都市 2024年",
  "theme": "都市奋斗与情感",
  "rules": "现实世界规则，科技发达，社交媒体盛行",
  "special": "玩家可以通过努力改变命运，金钱与人脉是核心资源",
  "atmosphere": "现实主义，偶有温情与戏剧性转折"
}
```

---

6. 后端 API 接口

所有接口均通过 tRPC 暴露，前端使用 `trpc.*.useQuery()` 或 `trpc.*.useMutation()` 调用。

6.1 auth 路由

Procedure	类型	权限	说明
`auth.me`	query	public	获取当前 Manus OAuth 登录用户信息
`auth.logout`	mutation	public	清除 Session Cookie，登出

6.2 account 路由（游戏账号）

Procedure	类型	权限	说明
`account.register`	mutation	public	注册游戏账号（用户名+密码，bcrypt 加密）
`account.login`	mutation	public	登录游戏账号，签发 JWT Cookie
`account.logout`	mutation	public	清除游戏账号 Session
`account.getSession`	query	public	获取当前游戏账号登录状态
`account.checkUsername`	query	public	检查用户名是否已被占用

register 输入参数：

```
{
  username: string,     // 2-20字符，支持中文/字母/数字/下划线
  password: string,     // 6-50字符
  displayName?: string // 可选显示名
}
```

register 返回值：

```
{
  success: true,
  userId: number,
  username: string,
  displayName: string | null
}
```

6.3 game 路由

Procedure	类型	权限	说明
`game.saveGame`	mutation	protected	保存当前游戏进度到云端
`game.loadGame`	query	protected	读取指定存档（需校验所有权）
`game.getLatestSave`	query	protected	获取最新存档（用于「继续旅程」）
`game.listSaves`	query	protected	列出所有存档（最多10条）
`game.deleteSave`	mutation	protected	删除指定存档
`game.aiProxy`	mutation	public	后端 AI 代理（无需 API Key）
`game.testApiConnection`	mutation	public	测试外部 API 连通性

aiProxy 输入参数：

```
{
  messages: Array<{ role: "system" | "user" | "assistant", content: string
}>,
  temperature?: number,    // 0-2，默认 0.85
  maxTokens?: number,      // 50-4000，默认 800
  schemaName?: string,     // JSON Schema 名称（结构化输出）
  schema?: Record<string, unknown>  // JSON Schema 定义
}
```

aiProxy 返回值：

```
{ success: true, content: string }
// 或
{ success: false, content: "", error: string }
```

6.4 scripts 路由

Procedure	类型	权限	说明
`scripts.getTemplates`	query	public	获取内置剧本模板列表（4套）
`scripts.getGenres`	query	public	获取剧本类型标签列表
`scripts.listScripts`	query	public	列出用户的所有剧本
`scripts.createScript`	mutation	public	创建新剧本（每用户上限20个）
`scripts.activateScript`	mutation	public	激活指定剧本（同时取消其他激活）
`scripts.updateScript`	mutation	public	更新剧本信息
`scripts.deleteScript`	mutation	public	删除剧本
`scripts.getScriptSaves`	query	public	获取指定剧本下的存档列表

内置剧本模板（4套）：

模板名	类型	封面	世界观主题
都市丽人	现代都市	🌆	现代都市奋斗与情感
古风仙缘	古风仙侠	🏯	修仙世界，灵气复苏
星际漂流	科幻未来	🚀	2387年，星际联邦时代
悬案迷踪	悬疑推理	🔍	现代都市，神秘案件

---

7. 前端状态管理

7.1 GameContext 架构

游戏的所有状态通过 `GameContext`（基于 `useReducer`）集中管理，避免 prop drilling 并保证状态一致性。

```typescript
// 完整状态结构
interface GameState {
  onboardingStep: OnboardingStep;  // 当前流程步骤
  saveId: number | null;           // 云端存档 ID
  player: PlayerProfile | null;    // 玩家角色数据
  gameDate: GameDate;              // 游戏内时间
  round: number;                   // 当前回合数
  storyHistory: StoryMessage[];    // 故事历史（最近100条）
  isAiThinking: boolean;           // AI 思考状态
  aiStatusText: string;            // AI 状态文字
  npcs: NpcProfile[];              // NPC 列表
  activeChatNpcId: number | null;  // 当前聊天 NPC
  chatMessages: Record<number, ChatMessage[]>; // 聊天记录
  inventory: InventoryItem[];      // 背包物品
  notes: Note[];                   // 记事本
  apiConfig: ApiConfig;            // AI API 配置
  appearanceConfig: AppearanceConfig; // 外观主题配置
  currentPage: GamePage;           // 当前页面
  scriptData: Record<string, unknown>; // 当前剧本设定
  loggedInUsername: string | null; // 游戏账号用户名
  loggedInUserId: number | null;   // 游戏账号 ID
}
```

7.2 OnboardingStep 流程

玩家进入游戏后经历以下步骤：

```
welcome → login → api-config → appearance → char-setup → main
  欢迎页     登录页    API配置页     外观设置页     角色创建页    主游戏
```

每个步骤均可通过 `goToStep(step)` 跳转，`LOAD_SAVE` action 会直接跳至 `main` 步骤。

7.3 核心 Action 类型

Action	说明
`SET_ONBOARDING`	切换流程步骤
`SET_PLAYER`	设置玩家角色
`ADD_STORY`	添加一条故事消息
`SET_AI_THINKING`	设置 AI 思考状态
`LOAD_SAVE`	从存档恢复完整状态
`SET_SCRIPT_DATA`	更新当前剧本设定
`RESET_STORY`	重置故事历史（切换剧本时使用）
`SET_LOGGED_IN_USER`	设置游戏账号登录状态
`ADVANCE_ROUND`	回合数 +1
`UPDATE_PLAYER_STATS`	更新玩家属性值（带 0-100 范围限制）

7.4 PlayerProfile 数据结构

```
interface PlayerProfile {
  name: string;
  gender: "male" | "female" | "other";
  age: number;
  zodiac: string;           // 星座
  background: string;       // 背景故事
  avatarUrl: string;
  location: string;         // 所在城市
  weather: string;          // 当前天气
  height: number;           // 身高(cm)
  weight: number;           // 体重(kg)
  bust?: number; waist?: number; hips?: number; // 三围（可选）
  skinTone: string;         // 肤色描述
  stats: PlayerStats;       // 七维属性
  moneyCny: number;         // 人民币余额
  moneyUsd: number;         // 美元余额
  moneyCoins: number;       // 游戏金币
  traits: string[];         // 性格标签
  talents: string[];        // 才能标签
  isVirgin: boolean;
  hasFirstKiss: boolean;
  isPregnant: boolean;
  pregnancyDays: number;
}

interface PlayerStats {
  health: number;    // 健康值 0-100
  energy: number;    // 精力值 0-100
  beauty: number;    // 魅力值 0-100
  physique: number;  // 体质值 0-100
  mood: number;      // 心情值 0-100
  stress: number;    // 压力值 0-100
  iq: number;        // 智力值 0-100
}
```

---

8. AI 引擎设计

8.1 双模式 AI 调用

`aiEngine.ts` 实现了两套 LLM 调用路径，自动根据配置选择：

```
用户配置了 API Key
    └── 直接 fetch → 用户指定的 OpenAI 兼容端点
用户未配置 / 开启内置代理
    └── fetch /api/trpc/game.aiProxy → 服务端 invokeLLM → Manus Built-in LLM
```

8.2 generateGameOpening（开场生成）

当玩家点击「开始游戏」时调用，生成 200-300 字的沉浸式开场白，并初始化 1-3 个 NPC。

输入： 玩家角色信息 + 剧本世界观设定 + API 配置

AI 输出 JSON 结构：

```
{
  "opening": "开场叙事文字（200-300字）",
  "sceneTitle": "场景标题（4-8字）",
  "choices": [
    {"id": "c1", "text": "选项文字", "hint": "影响暗示", "type": "action"},
    {"id": "c2", "text": "选项文字", "hint": "影响暗示", "type": "explore"},
    {"id": "c3", "text": "选项文字", "hint": "影响暗示", "type": "dialogue"}
  ],
  "initialNpcs": [
    {
      "name": "NPC名字",
      "gender": "female",
      "age": 25,
      "occupation": "职业",
      "relationship": "关系",
      "background": "背景",
      "traits": ["性格1", "性格2"],
      "affection": 50
    }
  ]
}
```

8.3 advanceStory（剧情推进）

每次玩家做出选择后调用，推进故事并生成下一回合的三个选项。

AI 推动机制：

触发条件	效果
回合数 % 4 == 0	强制引入意外转折（偶遇/发现秘密/突发危机/意外机遇）
回合数 % 7 == 0	引入新 NPC 角色
其他回合	正常剧情推进

AI 输出 JSON 结构：

```
{
  "sceneTitle": "场景标题（可选）",
  "narrative": "故事叙事（150-250字，第二人称）",
  "choices": [
    {"id": "c1", "text": "行动选项", "hint": "影响暗示", "statHint": "+心情",
"type": "action"},
    {"id": "c2", "text": "对话选项", "hint": "影响暗示", "statHint": "-精力",
"type": "dialogue"},
    {"id": "c3", "text": "休息选项", "hint": "影响暗示", "statHint": "+精力",
"type": "rest"}
  ],
  "statChanges": {
    "health": 0, "energy": -5, "mood": 10, "stress": -5,
    "beauty": 0, "physique": 0, "iq": 0, "moneyCny": 0
  },
  "timeAdvance": 2,
  "mood": "场景氛围关键词",
  "event": "特殊事件标签或null",
  "newNpc": null
}
```

8.4 选项类型系统

类型	标签	说明
`action`	行动	主动出击类选项
`dialogue`	对话	与人交流类选项
`explore`	探索	调查/发现类选项
`rest`	休息	恢复精力类选项
`special`	特殊	特殊事件/隐藏选项

8.5 System Prompt 构建

`buildSystemPrompt()` 函数根据以下信息动态构建 System Prompt：

- 玩家角色的完整人设（姓名/年龄/性格/背景/属性）
- 剧本世界观设定（时代/主题/规则/特殊设定/氛围）

- 输出语言（支持简体中文/繁体中文/英文/日文/韩文）
- R18 模式开关（影响内容尺度）

---

9. 主题系统

9.1 三套主题

主题名	标识符	主色调	风格
少女元氣	`sakura-girl`	桃粉 #E6A6AC + 翠绿 #A1C2B1	清新少女，花瓣粒子
橘子汽水	`orange-soda`	质感灰 #454545 + 活力橙 #FF6000	现代活力，气泡粒子
高贵黑金	`noble-black-gold`	深邃黑 #111A19 + 典雅金 #BB6830	古典奢华，金粉粒子

9.2 CSS 变量体系

每套主题通过 `.theme-*` class 覆盖以下 CSS 变量：

```
--theme-bg        /* 主背景色 */
--theme-bg2       /* 次背景色 */
--theme-card      /* 卡片背景 */
--theme-primary   /* 主色调 */
--theme-accent1   /* 强调色1 */
--theme-accent2   /* 强调色2 */
--theme-text1     /* 主文字色 */
--theme-text2     /* 次文字色 */
--theme-text3     /* 辅助文字色 */
--theme-border    /* 边框色 */
--theme-shadow    /* 阴影色 */
```

9.3 字体系统

字体标识	字体族	风格
`cinzel-elegant`	Cinzel + Noto Serif TC	古典优雅
`cormorant-romantic`	Cormorant Garamond + Noto Serif TC	浪漫文学
`playfair-classic`	Playfair Display + Noto Serif TC	经典衬线
`noto-minimal`	Noto Sans TC	现代简约

---

10. 用户流程

10.1 新用户完整流程

```
1. 访问首页（WelcomePage）
   ├── 已登录 → 显示「继续旅程」按钮（读取最新存档）
   └── 未登录 → 显示「开始你的故事」按钮

2. 登录/注册（LoginPage）
   ├── 游戏账号注册：输入用户名+密码 → 服务端 bcrypt 加密 → 写入 game_accounts
   ├── 游戏账号登录：验证密码 → 签发 JWT Cookie
   └── Manus OAuth：跳转 OAuth 流程

3. API 配置（ApiConfigPage）
   ├── 开启内置代理（默认）→ 无需配置，直接下一步
   └── 自定义 API：填写端点/Key/模型 → 测试连通性

4. 外观设置（AppearancePage）
   └── 选择主题/字体/语言/UI 密度

5. 角色创建（SetupPage，6步向导）
   ├── 基础信息（姓名/性别/年龄/星座）
   ├── 外貌设定（身高/体重/肤色/三围）
   ├── 背景故事（出生地/背景/位置/天气）
   ├── 性格才能（性格标签/才能标签）
   ├── 初始 NPC（关系人设定）
   └── 确认创建

6. 主游戏（MainGame）
   ├── 点击「开始游戏」→ AI 生成开场剧情
   ├── 阅读故事 → 选择三选一选项或自由输入
   ├── AI 推进故事 → 更新属性 → 自动存档
   └── 底部导航：故事/社交/剧本/背包/设置
```

---

11. 多剧本系统

11.1 设计理念

多剧本系统允许玩家在同一账号下维护多个平行故事线。每个剧本有独立的世界观设定，切换剧本时故事历史会重置，但玩家角色数据保留。

11.2 ScriptPage 功能

ScriptPage 提供三种视图模式：

视图	功能
`list`	显示所有剧本卡片，支持激活/删除/查看详情
`create`	创建新剧本（模板选择或自定义）
`detail`	查看剧本详情（世界观/游玩记录/关联存档）

11.3 剧本切换流程

```
用户点击「使用此剧本」
    ↓
activateScript mutation（更新数据库 isActive 字段）
    ↓
dispatch SET_SCRIPT_DATA（将 worldSetting 注入 GameContext）
    ↓
dispatch RESET_STORY（清空故事历史，round 重置为 1）
    ↓
dispatch SET_ONBOARDING("main")（进入主游戏）
    ↓
StoryPage 检测到 storyHistory 为空 → 显示「开始游戏」按钮
    ↓
玩家点击「开始游戏」→ generateGameOpening（携带新剧本世界观）
```

---

12. 云端存档系统

12.1 自动存档机制

每次 AI 推进故事后（`advanceStory` 完成），`StoryPage` 会自动调用 `useSaveGame().saveGame("auto")` 进行静默存档。存档使用防重入锁（`isSavingRef`）避免并发写入。

12.2 存档数据序列化

所有复杂对象（PlayerProfile、NpcProfile[]、StoryMessage[] 等）均序列化为 JSON 字符串存入 TEXT 字段。读取时反序列化，并对 `Date` 类型进行特殊处理（`new Date(m.timestamp)`）。

12.3 存档隔离

`game.saveGame` 和 `game.loadGame` 均为 `protectedProcedure`，通过 `ctx.user.id` 确保用户只能访问自己的存档。删除存档时额外校验 `save.userId === ctx.user.id`。

---

13. 账号认证系统

13.1 双认证体系

认证方式	实现	Cookie 名	适用场景
Manus OAuth	`server/_core/oauth.ts`	`manus_session`	已有 Manus 账号的用户
游戏账号	`server/routers/account.ts`	`game_session`	不想绑定 Manus 的玩家

13.2 游戏账号安全设计

密码存储采用 bcrypt 算法（cost factor 12），每个账号生成独立的随机盐值。用户名验证规则：2-20 字符，支持中文/英文字母/数字/下划线，禁止纯数字用户名（防止与自动生成的随机数字 ID 混淆）。

若用户注册时未设置用户名，系统自动生成格式为 `user_XXXXXX` 的随机数字 ID（6位随机数），确保唯一性。

13.3 Session 管理

游戏账号登录后签发 JWT，存入 HttpOnly Cookie。`account.getSession` 接口在每次页面加载时验证 Cookie 有效性，恢复登录状态到 `GameContext`。

---

14. 部署说明

14.1 环境变量

变量名	说明	必须
`DATABASE_URL`	MySQL/TiDB 连接字符串	✓
`JWT_SECRET`	Session Cookie 签名密钥	✓
`BUILT_IN_FORGE_API_KEY`	Manus 内置 LLM API Key	✓
`BUILT_IN_FORGE_API_URL`	Manus 内置 LLM 端点	✓
`VITE_APP_ID`	Manus OAuth 应用 ID	✓
`OAUTH_SERVER_URL`	Manus OAuth 后端地址	✓
`VITE_OAUTH_PORTAL_URL`	Manus 登录门户地址	✓
`OWNER_OPEN_ID`	项目所有者 Open ID	—
`OWNER_NAME`	项目所有者名称	—

14.2 启动命令

```
# 安装依赖
pnpm install

# 推送数据库 schema
pnpm db:push

# 开发模式
pnpm dev

# 生产构建
pnpm build

# 运行测试
pnpm test
```

14.3 测试覆盖

测试文件	测试数	覆盖范围
`server/scripts.test.ts`	6	剧本模板/类型/路由结构
`server/auth.logout.test.ts`	1	登出 Cookie 清除

文档版本：v1.0.0 | 最后更新：2026-02-20 | 原创作品，版权归悠然若妃所有

Chuyue（初悦）— 核心代码完整文档

> ***版本：**v1.0.0 | **日期：**2026-02-20 | **原创作品，版权归悠然若妃所有***

本文档收录项目所有核心文件的完整代码，并附有详细注释说明，供开发者参考与二次开发使用。

目录

1. 数据库 Schema（drizzle/schema.ts）
2. 主路由聚合（server/routers.ts）
3. 游戏存档路由（server/routers/game.ts）
4. 账号认证路由（server/routers/account.ts）
5. 剧本管理路由（server/routers/scripts.ts）
6. 全局状态管理（client/src/contexts/GameContext.tsx）
7. AI 引擎（client/src/lib/aiEngine.ts）
8. 云端存档 Hook（client/src/hooks/useSaveGame.ts）
9. 单元测试（server/scripts.test.ts）

---

1. 数据库 Schema

文件路径： `drizzle/schema.ts`

本文件定义了游戏的全部数据库表结构，使用 Drizzle ORM 的类型安全 DSL 编写。共四张表：用户表、游戏账号表、游戏存档表、剧本表。

```
import { int, mysqlEnum, mysqlTable, text, timestamp, varchar, boolean }
from "drizzle-orm/mysql-core";

// ═══════════════════════════════════════════════════════
// users 表 — Manus OAuth 用户
// 通过 Manus OAuth 登录的用户信息存储于此
// ═══════════════════════════════════════════════════════
export const users = mysqlTable("users", {
  id: int("id").autoincrement().primaryKey(),
  openId: varchar("openId", { length: 64 }).notNull().unique(), // Manus
OAuth 唯一标识
  name: text("name"),
  email: varchar("email", { length: 320 }),
  loginMethod: varchar("loginMethod", { length: 64 }),
  role: mysqlEnum("role", ["user", "admin"]).default("user").notNull(),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
  lastSignedIn: timestamp("lastSignedIn").defaultNow().notNull(),
});
export type User = typeof users.$inferSelect;
export type InsertUser = typeof users.$inferInsert;

// ═══════════════════════════════════════════════════════
// game_saves 表 — 游戏存档
// 储存玩家的完整游戏进度，支持多存档槽
// 所有复杂对象均序列化为 JSON 字符串存入 TEXT 字段
// ═══════════════════════════════════════════════════════
export const gameSaves = mysqlTable("game_saves", {
  id: int("id").autoincrement().primaryKey(),
  userId: int("userId").notNull(),
  slotName: varchar("slotName", { length: 64 }).default("auto"),
  scriptId: int("scriptId"),              // 关联 gameScripts 表
  playerData: text("playerData"),         // 玩家角色 JSON
  npcsData: text("npcsData"),             // NPC 列表 JSON
  storyData: text("storyData"),           // 故事历史 JSON（最近50条）
  inventoryData: text("inventoryData"),   // 背包 JSON
  notesData: text("notesData"),           // 记事本 JSON
  chatData: text("chatData"),             // 聊天记录 JSON
  apiConfigData: text("apiConfigData"),   // API 配置 JSON（不含 key）
  appearanceData: text("appearanceData"),// 外观配置 JSON
  scriptData: text("scriptData"),         // 剧本设定 JSON
  round: int("round").default(0),
  gameYear: int("gameYear").default(2024),
  gameMonth: int("gameMonth").default(1),
  gameDay: int("gameDay").default(1),
```

```
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
});
export type GameSave = typeof gameSaves.$inferSelect;
export type InsertGameSave = typeof gameSaves.$inferInsert;

// ═══════════════════════════════════════════════════════
// game_accounts 表 — 游戏独立账号
// 与 Manus OAuth 并行，玩家可选择任意一种登录方式
// 密码使用 bcrypt（cost factor 12）加密存储
// ═══════════════════════════════════════════════════════
export const gameAccounts = mysqlTable("game_accounts", {
  id: int("id").autoincrement().primaryKey(),
  username: varchar("username", { length: 32 }).notNull().unique(), // 唯一用户名
  passwordHash: varchar("password_hash", { length: 128 }).notNull(),
  salt: varchar("salt", { length: 32 }).notNull(),
  displayName: varchar("display_name", { length: 32 }),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
  lastSignedIn: timestamp("lastSignedIn").defaultNow().notNull(),
});
export type GameAccount = typeof gameAccounts.$inferSelect;
export type InsertGameAccount = typeof gameAccounts.$inferInsert;

// ═══════════════════════════════════════════════════════
// game_scripts 表 — 剧本配置
// 玩家可创建多个剧本，每个剧本有独立的世界观设定
// isActive 字段标记当前激活的剧本（每用户只有一个）
// ═══════════════════════════════════════════════════════
export const gameScripts = mysqlTable("game_scripts", {
  id: int("id").autoincrement().primaryKey(),
  userId: int("userId").notNull(),
  name: varchar("name", { length: 64 }).notNull(),
  description: text("description"),
  genre: varchar("genre", { length: 32 }).default("现代"),
  worldSetting: text("worldSetting"),    // JSON：{era, theme, rules, special, atmosphere}
  coverColor: varchar("coverColor", { length: 16 }).default("#CBAF98"),
  coverIcon: varchar("coverIcon", { length: 8 }).default("📖"),
  isActive: boolean("isActive").default(false),
  totalRounds: int("totalRounds").default(0),
  lastPlayedAt: timestamp("lastPlayedAt").defaultNow(),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
});
```

```
export type GameScript = typeof gameScripts.$inferSelect;
export type InsertGameScript = typeof gameScripts.$inferInsert;
```

---

2. 主路由聚合

文件路径： `server/routers.ts`

将所有子路由聚合为一个 `appRouter`，并导出 `AppRouter` 类型供前端 tRPC 客户端使用。

```
import { COOKIE_NAME } from "@shared/const";
import { getSessionCookieOptions } from "./_core/cookies";
import { systemRouter } from "./_core/systemRouter";
import { publicProcedure, router } from "./_core/trpc";
import { gameRouter } from "./routers/game";
import { accountRouter } from "./routers/account";
import { scriptsRouter } from "./routers/scripts";

export const appRouter = router({
  // 系统通知路由（内置）
  system: systemRouter,

  // 游戏账号路由（独立注册/登录体系）
  account: accountRouter,

  // Manus OAuth 认证路由
  auth: router({
    // 获取当前登录用户信息（通过 Manus OAuth Cookie）
    me: publicProcedure.query(opts => opts.ctx.user),

    // 登出：清除 Session Cookie
    logout: publicProcedure.mutation(({ ctx }) => {
      const cookieOptions = getSessionCookieOptions(ctx.req);
      ctx.res.clearCookie(COOKIE_NAME, { ...cookieOptions, maxAge: -1 });
      return { success: true } as const;
    }),
  }),

  // 游戏核心路由（存档、AI代理）
  game: gameRouter,

  // 剧本管理路由
  scripts: scriptsRouter,
});

// 导出类型供前端 tRPC 客户端推断
export type AppRouter = typeof appRouter;
```

---

3. 游戏存档路由

文件路径： `server/routers/game.ts`

处理游戏存档的 CRUD 操作，以及后端 AI 代理（让前端无需配置 API Key 即可使用 AI）。

```
import { z } from "zod";
import { eq, desc } from "drizzle-orm";
import { TRPCError } from "@trpc/server";
import { protectedProcedure, publicProcedure, router } from "../_core/trpc";
import { getDb } from "../db";
import { gameSaves } from "../../drizzle/schema";
import { invokeLLM } from "../_core/llm";

export const gameRouter = router({
  /**
   * 保存游戏进度（需要登录）
   * 支持 upsert：有 saveId 则更新，否则新建
   * 每用户最多保留 10 个存档槽
   */
  saveGame: protectedProcedure
    .input(z.object({
      saveId: z.number().optional(),
      slotName: z.string().default("auto"),
      scriptId: z.number().optional(),
      playerData: z.string().optional(),
      npcsData: z.string().optional(),
      storyData: z.string().optional(),
      chatData: z.string().optional(),
      inventoryData: z.string().optional(),
      notesData: z.string().optional(),
      scriptData: z.string().optional(),
      round: z.number().default(0),
      gameYear: z.number().default(2024),
      gameMonth: z.number().default(1),
      gameDay: z.number().default(1),
    }))
    .mutation(async ({ ctx, input }) => {
      const db = await getDb();
      if (!db) throw new TRPCError({ code: "INTERNAL_SERVER_ERROR" });

      // 如果传入了 saveId，先校验所有权再更新
      if (input.saveId) {
        const existing = await db.select().from(gameSaves)
          .where(eq(gameSaves.id, input.saveId)).limit(1);
        if (existing[0] && existing[0].userId === ctx.user.id) {
          await db.update(gameSaves).set({
            slotName: input.slotName,
            scriptId: input.scriptId ?? null,
            playerData: input.playerData ?? null,
            npcsData: input.npcsData ?? null,
```

```
            storyData: input.storyData ?? null,
            chatData: input.chatData ?? null,
            inventoryData: input.inventoryData ?? null,
            notesData: input.notesData ?? null,
            scriptData: input.scriptData ?? null,
            round: input.round,
            gameYear: input.gameYear,
            gameMonth: input.gameMonth,
            gameDay: input.gameDay,
          }).where(eq(gameSaves.id, input.saveId));
          return { saveId: input.saveId, isNew: false };
        }
      }

      // 新建存档
      const result = await db.insert(gameSaves).values({
        userId: ctx.user.id,
        slotName: input.slotName,
        scriptId: input.scriptId ?? null,
        playerData: input.playerData ?? null,
        npcsData: input.npcsData ?? null,
        storyData: input.storyData ?? null,
        chatData: input.chatData ?? null,
        inventoryData: input.inventoryData ?? null,
        notesData: input.notesData ?? null,
        scriptData: input.scriptData ?? null,
        round: input.round,
        gameYear: input.gameYear,
        gameMonth: input.gameMonth,
        gameDay: input.gameDay,
      });
      return {
        saveId: Number((result as unknown as { insertId: number
}).insertId),
        isNew: true
      };
    }),

  /** 读取单个存档（需要登录，校验所有权） */
  loadGame: protectedProcedure
    .input(z.object({ saveId: z.number() }))
    .query(async ({ ctx, input }) => {
      const db = await getDb();
      if (!db) throw new Error("Database unavailable");
      const save = await db.select().from(gameSaves)
        .where(eq(gameSaves.id, input.saveId)).limit(1);
```

```
    if (!save[0] || save[0].userId !== ctx.user.id) {
      throw new Error("Save not found or unauthorized");
    }
    return save[0];
  }),

/** 读取最新存档（用于「继续旅程」按钮） */
getLatestSave: protectedProcedure.query(async ({ ctx }) => {
  const db = await getDb();
  if (!db) return null;
  const saves = await db.select().from(gameSaves)
    .where(eq(gameSaves.userId, ctx.user.id))
    .orderBy(desc(gameSaves.updatedAt)).limit(1);
  return saves[0] ?? null;
}),

/** 列出所有存档（最多10条，按更新时间倒序） */
listSaves: protectedProcedure.query(async ({ ctx }) => {
  const db = await getDb();
  if (!db) return [];
  return await db.select().from(gameSaves)
    .where(eq(gameSaves.userId, ctx.user.id))
    .orderBy(desc(gameSaves.updatedAt)).limit(10);
}),

/** 删除存档（校验所有权） */
deleteSave: protectedProcedure
  .input(z.object({ saveId: z.number() }))
  .mutation(async ({ ctx, input }) => {
    const db = await getDb();
    if (!db) throw new TRPCError({ code: "INTERNAL_SERVER_ERROR" });
    const save = await db.select().from(gameSaves)
      .where(eq(gameSaves.id, input.saveId)).limit(1);
    if (!save[0] || save[0].userId !== ctx.user.id) {
      throw new Error("Save not found or unauthorized");
    }
    await db.delete(gameSaves).where(eq(gameSaves.id, input.saveId));
    return { success: true };
  }),

/**
 * 后端 AI 代理（公开接口，无需登录）
 * ────────────────────────────────────────────────────────
 * 使用服务器内置 LLM（BUILT_IN_FORGE_API_KEY），
 * 用户无需填写任何 API Key 即可运行游戏。
 * 前端通过 trpc.game.aiProxy.mutate() 调用。
```

```
 * 支持结构化 JSON 输出（传入 schema 参数时启用）。
 * ──────────────────────────────────────────────────────
 */
aiProxy: publicProcedure
  .input(z.object({
    messages: z.array(z.object({
      role: z.enum(["system", "user", "assistant"]),
      content: z.string(),
    })),
    temperature: z.number().min(0).max(2).optional().default(0.85),
    maxTokens: z.number().min(50).max(4000).optional().default(800),
    schemaName: z.string().optional(),
    schema: z.record(z.string(), z.unknown()).optional(),
  }))
  .mutation(async ({ input }) => {
    try {
      const params: Parameters<typeof invokeLLM>[0] = {
        messages: input.messages,
        max_tokens: input.maxTokens,
      };
      // 有 JSON Schema 时使用结构化输出，否则使用 json_object 模式
      if (input.schema && input.schemaName) {
        params.response_format = {
          type: "json_schema",
          json_schema: {
            name: input.schemaName,
            strict: true,
            schema: input.schema as Record<string, unknown>,
          },
        };
      } else {
        params.response_format = { type: "json_object" };
      }
      const result = await invokeLLM(params);
      const content = result.choices?.[0]?.message?.content ?? "";
      return { success: true as const, content };
    } catch (e: unknown) {
      const msg = e instanceof Error ? e.message : "Unknown error";
      return { success: false as const, content: "", error: msg };
    }
  }),

/** 测试外部 API 连通性（前端直接调用，key 不经过服务器） */
testApiConnection: publicProcedure
  .input(z.object({
    endpoint: z.string(),
```

```
      apiKey: z.string(),
      model: z.string(),
    }))
    .mutation(async ({ input }) => {
      try {
        const response = await fetch(`${input.endpoint}/chat/completions`, {
          method: "POST",
          headers: {
            "Content-Type": "application/json",
            Authorization: `Bearer ${input.apiKey}`,
          },
          body: JSON.stringify({
            model: input.model,
            messages: [{ role: "user", content: "Hello! Reply with 'OK'
only." }],
            max_tokens: 10,
          }),
          signal: AbortSignal.timeout(10000),
        });
        if (!response.ok) {
          const errText = await response.text();
          return { success: false, error: `HTTP ${response.status}:
${errText.slice(0, 200)}` };
        }
        const data = await response.json() as {
          choices?: Array<{ message?: { content?: string } }>
        };
        const reply = data?.choices?.[0]?.message?.content || "No response";
        return { success: true, reply };
      } catch (e: unknown) {
        const msg = e instanceof Error ? e.message : "Unknown error";
        return { success: false, error: msg };
      }
    }),
});
```

---

4. 账号认证路由

文件路径： `server/routers/account.ts`

实现游戏独立账号体系：注册、登录、登出、会话查询、用户名检查。密码使用 bcrypt 加密，会话使用 JWT Cookie。

```
import { z } from "zod";
import bcrypt from "bcryptjs";
import * as jose from "jose";
import { TRPCError } from "@trpc/server";
import { eq } from "drizzle-orm";
import { publicProcedure, protectedProcedure, router } from "../_core/trpc";
import { getDb } from "../db";
import { gameAccounts } from "../../drizzle/schema";
import type { TrpcContext } from "../_core/context";

// ── 常量配置 ──
const SALT_ROUNDS = 12;                    // bcrypt 加密强度
const GAME_SESSION_COOKIE = "game_session"; // Cookie 名称
const JWT_EXPIRY = "30d";                  // JWT 有效期 30 天

// 用户名规则：2-16字符，支持中文/英文/数字/下划线，不能纯数字
const USERNAME_REGEX = /^(?!\d+$)[\u4e00-\u9fa5a-zA-Z0-9_]{2,16}$/;

// ── JWT 辅助函数 ──
type Ctx = { req: { cookies: Record<string, string> }; res: { cookie:
Function; clearCookie: Function } };

async function getJwtSecret(): Promise<Uint8Array> {
  const secret = process.env.JWT_SECRET || "fallback-secret-change-in-
production";
  return new TextEncoder().encode(secret);
}

async function signToken(payload: { id: number; username: string }):
Promise<string> {
  const secret = await getJwtSecret();
  return new jose.SignJWT({ ...payload })
    .setProtectedHeader({ alg: "HS256" })
    .setExpirationTime(JWT_EXPIRY)
    .setIssuedAt()
    .sign(secret);
}

async function verifyToken(token: string): Promise<{ id: number; username:
string } | null> {
  try {
    const secret = await getJwtSecret();
    const { payload } = await jose.jwtVerify(token, secret);
    return payload as { id: number; username: string };
  } catch {
```

```
    return null;
  }
}

function setCookie(ctx: Ctx, token: string) {
  ctx.res.cookie(GAME_SESSION_COOKIE, token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === "production",
    sameSite: "lax",
    maxAge: 30 * 24 * 60 * 60 * 1000, // 30天
    path: "/",
  });
}

function clearCookie(ctx: Ctx) {
  ctx.res.clearCookie(GAME_SESSION_COOKIE, { path: "/" });
}

function getToken(ctx: Ctx): string | null {
  return ctx.req.cookies?.[GAME_SESSION_COOKIE] ?? null;
}

// ── 随机用户名生成器 ──
// 格式：user_XXXXXX（6位随机数字）
function randomUsername(): string {
  const num = Math.floor(100000 + Math.random() * 900000);
  return `user_${num}`;
}

export const accountRouter = router({
  /**
   * 注册游戏账号
   * - username 可选：若不填则自动生成 user_XXXXXX 格式
   * - 密码使用 bcrypt（cost 12）加密
   * - 注册成功后自动签发 JWT Cookie，无需再次登录
   */
  register: publicProcedure
    .input(z.object({
      username: z.string().max(16).optional(),
      password: z.string().min(6, "密码至少6位").max(50),
      displayName: z.string().max(32).optional(),
    }))
    .mutation(async ({ ctx, input }) => {
      const db = await getDb();
      if (!db) throw new TRPCError({ code: "INTERNAL_SERVER_ERROR", message:
"数据库不可用" });
```

```
// 确定用户名（为空则自动生成，确保不重复）
let username = (input.username ?? "").trim();
if (!username) {
  let attempts = 0;
  while (attempts < 10) {
    const candidate = randomUsername();
    const exists = await db.select({ id: gameAccounts.id })
      .from(gameAccounts)
      .where(eq(gameAccounts.username, candidate)).limit(1);
    if (!exists[0]) { username = candidate; break; }
    attempts++;
  }
  if (!username) throw new TRPCError({
    code: "INTERNAL_SERVER_ERROR",
    message: "用户名生成失败，请手动填写"
  });
} else {
  // 验证用户名格式
  if (!USERNAME_REGEX.test(username)) {
    throw new TRPCError({
      code: "BAD_REQUEST",
      message: "用户名只能包含中文、英文、数字或下划线，长度 2-16 字符",
    });
  }
  // 检查是否已被注册
  const existing = await db.select({ id: gameAccounts.id })
    .from(gameAccounts)
    .where(eq(gameAccounts.username, username)).limit(1);
  if (existing[0]) {
    throw new TRPCError({
      code: "CONFLICT",
      message: `用户名「${username}」已被注册，请换一个`
    });
  }
}

// 哈希密码并写入数据库
const salt = await bcrypt.genSalt(SALT_ROUNDS);
const passwordHash = await bcrypt.hash(input.password, salt);
const result = await db.insert(gameAccounts).values({
  username,
  passwordHash,
  salt,
  displayName: input.displayName ?? username,
});
```

```
      const newId = Number((result as unknown as { insertId: number
}).insertId);

      // 注册成功，自动签发 JWT
      const token = await signToken({ id: newId, username });
      setCookie(ctx as unknown as Ctx, token);

      return {
        success: true,
        userId: newId,
        username,
        displayName: input.displayName ?? username,
        message: `注册成功！欢迎，${input.displayName ?? username}`,
      };
    }),

  /**
   * 登录游戏账号
   * - 验证用户名和密码（bcrypt.compare）
   * - 登录成功后更新 lastSignedIn 并签发 JWT Cookie
   */
  login: publicProcedure
    .input(z.object({
      username: z.string().min(1, "请输入用户名"),
      password: z.string().min(1, "请输入密码"),
    }))
    .mutation(async ({ ctx, input }) => {
      const db = await getDb();
      if (!db) throw new TRPCError({ code: "INTERNAL_SERVER_ERROR", message:
"数据库不可用" });

      const accounts = await db.select().from(gameAccounts)
        .where(eq(gameAccounts.username, input.username.trim())).limit(1);
      if (!accounts[0]) {
        throw new TRPCError({ code: "UNAUTHORIZED", message: "用户名不存在"
});
      }

      const account = accounts[0];
      const valid = await bcrypt.compare(input.password,
account.passwordHash);
      if (!valid) {
        throw new TRPCError({ code: "UNAUTHORIZED", message: "密码错误" });
      }

      // 更新最后登录时间
```

```
      await db.update(gameAccounts)
        .set({ lastSignedIn: new Date() })
        .where(eq(gameAccounts.id, account.id));

      const token = await signToken({ id: account.id, username:
account.username });
      setCookie(ctx as unknown as Ctx, token);

      return {
        success: true,
        userId: account.id,
        username: account.username,
        displayName: account.displayName ?? account.username,
        message: `登录成功！欢迎回来，${account.displayName ??
account.username}`,
      };
    }),

  /** 登出：清除 game_session Cookie */
  logout: publicProcedure.mutation(async ({ ctx }) => {
    clearCookie(ctx as unknown as Ctx);
    return { success: true };
  }),

  /**
   * 获取当前游戏账号会话
   * 通过 game_session Cookie 中的 JWT 验证身份
   * 返回账号信息（不含密码）
   */
  getSession: publicProcedure.query(async ({ ctx }) => {
    const token = getToken(ctx as unknown as Ctx);
    if (!token) return null;
    const payload = await verifyToken(token);
    if (!payload) return null;
    const db = await getDb();
    if (!db) return null;
    const accounts = await db.select({
      id: gameAccounts.id,
      username: gameAccounts.username,
      displayName: gameAccounts.displayName,
      createdAt: gameAccounts.createdAt,
      lastSignedIn: gameAccounts.lastSignedIn,
    }).from(gameAccounts)
      .where(eq(gameAccounts.id, payload.id)).limit(1);
    return accounts[0] ?? null;
  }),
```

```
  /**
   * 检查用户名是否可用（实时验证，用于注册表单）
   * - 空用户名：返回「将自动生成随机用户名」
   * - 格式不符：返回错误说明
   * - 已被注册：返回冲突提示
   */
  checkUsername: publicProcedure
    .input(z.object({ username: z.string() }))
    .query(async ({ input }) => {
      const username = input.username.trim();
      if (!username) return { available: true, message: "将自动生成随机用户名"
};
      if (!USERNAME_REGEX.test(username)) {
        return { available: false, message: "用户名只能包含中文、英文、数字或下划
线，长度 2-16 字符" };
      }
      const db = await getDb();
      if (!db) return { available: true, message: "" };
      const existing = await db.select({ id: gameAccounts.id })
        .from(gameAccounts)
        .where(eq(gameAccounts.username, username)).limit(1);
      if (existing[0]) {
        return { available: false, message: `「${username}」已被注册` };
      }
      return { available: true, message: "用户名可用 ✓" };
    }),
});
```

---

5. 剧本管理路由

文件路径： `server/routers/scripts.ts`

管理玩家的剧本（创建/激活/更新/删除），提供内置模板和类型标签。

```
import { z } from "zod";
import { eq, desc, and } from "drizzle-orm";
import { TRPCError } from "@trpc/server";
import { publicProcedure, router } from "../_core/trpc";
import { getDb } from "../db";
import { gameScripts, gameSaves } from "../../drizzle/schema";

// ── 内置剧本类型标签 ──
const GENRES = [
  "现代都市", "古风仙侠", "科幻未来", "悬疑推理",
  "奇幻冒险", "末世求生", "校园青春", "职场商战",
  "宫廷权谋", "武侠江湖", "克苏鲁", "自定义",
];

// ── 内置剧本模板（4套） ──
const SCRIPT_TEMPLATES = [
  {
    name: "都市丽人",
    genre: "现代都市",
    description: "在繁华都市中追逐梦想，经历爱恨情仇，书写属于你的都市传奇。",
    coverColor: "#E6A6AC",
    coverIcon: "🌆",
    worldSetting: JSON.stringify({
      era: "现代都市 2024年",
      theme: "都市奋斗与情感",
      rules: "现实世界规则，科技发达，社交媒体盛行",
      special: "玩家可以通过努力改变命运，金钱与人脉是核心资源",
      atmosphere: "现实主义，偶有温情与戏剧性转折",
    }),
  },
  {
    name: "古风仙缘",
    genre: "古风仙侠",
    description: "修仙问道，寻觅仙缘，在灵气复苏的世界中踏上飞升之路。",
    coverColor: "#9DBFC3",
    coverIcon: "🏯",
    worldSetting: JSON.stringify({
      era: "架空古代，灵气复苏时代",
      theme: "修仙与情缘",
      rules: "灵气修炼体系，境界从练气到渡劫，门派林立",
      special: "天材地宝、奇遇秘境、仙缘际会，命运由修为与机缘共同决定",
      atmosphere: "古典唯美，仙气飘渺，情深缘浅",
    }),
  },
  {
```

```
    name: "星际漂流",
    genre: "科幻未来",
    description: "2387年，星际联邦时代。驾驶飞船穿越星系，探索未知文明。",
    coverColor: "#6B8CAE",
    coverIcon: "🚀",
    worldSetting: JSON.stringify({
      era: "2387年，星际联邦时代",
      theme: "星际探索与文明冲突",
      rules: "超光速飞行已实现，AI 与人类共存，多星系政治博弈",
      special: "异星文明、量子意识、时空折叠，科技即是魔法",
      atmosphere: "宏大壮阔，孤独与希望并存",
    }),
  },
  {
    name: "悬案迷踪",
    genre: "悬疑推理",
    description: "现代都市中的神秘案件接连发生，真相隐藏在重重迷雾之后。",
    coverColor: "#454545",
    coverIcon: "🔍",
    worldSetting: JSON.stringify({
      era: "现代都市，暗流涌动",
      theme: "悬疑推理与人性探索",
      rules: "现实世界规则，但存在隐秘组织和未解之谜",
      special: "每个人都有秘密，真相往往出人意料，信任是最稀缺的资源",
      atmosphere: "紧张压抑，悬念迭起，黑色幽默",
    }),
  },
];

export const scriptsRouter = router({
  /** 获取内置剧本模板列表 */
  getTemplates: publicProcedure.query(() => SCRIPT_TEMPLATES),

  /** 获取剧本类型标签列表 */
  getGenres: publicProcedure.query(() => GENRES),

  /** 列出用户的所有剧本（按最后游玩时间倒序） */
  listScripts: publicProcedure
    .input(z.object({ userId: z.number() }))
    .query(async ({ input }) => {
      const db = await getDb();
      if (!db) return [];
      return await db.select().from(gameScripts)
        .where(eq(gameScripts.userId, input.userId))
        .orderBy(desc(gameScripts.lastPlayedAt)).limit(20);
    }),
```

```
  /**
   * 创建新剧本
   * - 每用户最多 20 个剧本
   * - worldSetting 为 JSON 字符串（{era, theme, rules, special, atmosphere}）
   */
  createScript: publicProcedure
    .input(z.object({
      userId: z.number(),
      name: z.string().min(1).max(64),
      description: z.string().max(500).optional(),
      genre: z.string().optional().default("现代都市"),
      worldSetting: z.string().optional(),
      coverColor: z.string().optional().default("#CBAF98"),
      coverIcon: z.string().optional().default("📖"),
    }))
    .mutation(async ({ input }) => {
      const db = await getDb();
      if (!db) throw new TRPCError({ code: "INTERNAL_SERVER_ERROR", message:
"数据库不可用" });

      // 检查数量限制
      const existing = await db.select({ id: gameScripts.id })
        .from(gameScripts).where(eq(gameScripts.userId, input.userId));
      if (existing.length >= 20) {
        throw new TRPCError({
          code: "BAD_REQUEST",
          message: "剧本数量已达上限（20个），请删除旧剧本后再创建"
        });
      }

      const result = await db.insert(gameScripts).values({
        userId: input.userId,
        name: input.name,
        description: input.description ?? null,
        genre: input.genre,
        worldSetting: input.worldSetting ?? null,
        coverColor: input.coverColor,
        coverIcon: input.coverIcon,
        isActive: false,
        totalRounds: 0,
        lastPlayedAt: new Date(),
      });
      return {
        scriptId: Number((result as unknown as { insertId: number
}).insertId),
```

```
        success: true
      };
    }),

  /**
   * 激活剧本（切换当前使用的剧本）
   * 先取消该用户所有剧本的激活状态，再激活指定剧本
   */
  activateScript: publicProcedure
    .input(z.object({ userId: z.number(), scriptId: z.number() }))
    .mutation(async ({ input }) => {
      const db = await getDb();
      if (!db) throw new TRPCError({ code: "INTERNAL_SERVER_ERROR", message:
"数据库不可用" });

      // 取消所有激活
      await db.update(gameScripts).set({ isActive: false })
        .where(eq(gameScripts.userId, input.userId));

      // 激活指定剧本
      await db.update(gameScripts)
        .set({ isActive: true, lastPlayedAt: new Date() })
        .where(and(
          eq(gameScripts.id, input.scriptId),
          eq(gameScripts.userId, input.userId)
        ));
      return { success: true };
    }),

  /** 更新剧本信息（仅更新传入的字段） */
  updateScript: publicProcedure
    .input(z.object({
      userId: z.number(),
      scriptId: z.number(),
      name: z.string().min(1).max(64).optional(),
      description: z.string().max(500).optional(),
      genre: z.string().optional(),
      worldSetting: z.string().optional(),
      coverColor: z.string().optional(),
      coverIcon: z.string().optional(),
      totalRounds: z.number().optional(),
    }))
    .mutation(async ({ input }) => {
      const db = await getDb();
      if (!db) throw new TRPCError({ code: "INTERNAL_SERVER_ERROR", message:
"数据库不可用" });
```

```
      const updateData: Partial<typeof gameScripts.$inferInsert> = {};
      if (input.name !== undefined) updateData.name = input.name;
      if (input.description !== undefined) updateData.description =
input.description;
      if (input.genre !== undefined) updateData.genre = input.genre;
      if (input.worldSetting !== undefined) updateData.worldSetting =
input.worldSetting;
      if (input.coverColor !== undefined) updateData.coverColor =
input.coverColor;
      if (input.coverIcon !== undefined) updateData.coverIcon =
input.coverIcon;
      if (input.totalRounds !== undefined) updateData.totalRounds =
input.totalRounds;

      await db.update(gameScripts).set(updateData)
        .where(and(
          eq(gameScripts.id, input.scriptId),
          eq(gameScripts.userId, input.userId)
        ));
      return { success: true };
    }),

  /** 删除剧本（同时不删除关联存档，存档独立保留） */
  deleteScript: publicProcedure
    .input(z.object({ userId: z.number(), scriptId: z.number() }))
    .mutation(async ({ input }) => {
      const db = await getDb();
      if (!db) throw new TRPCError({ code: "INTERNAL_SERVER_ERROR", message:
"数据库不可用" });
      await db.delete(gameScripts)
        .where(and(
          eq(gameScripts.id, input.scriptId),
          eq(gameScripts.userId, input.userId)
        ));
      return { success: true };
    }),

  /** 获取指定剧本下的存档列表 */
  getScriptSaves: publicProcedure
    .input(z.object({ userId: z.number(), scriptId: z.number() }))
    .query(async ({ input }) => {
      const db = await getDb();
      if (!db) return [];
      return await db.select().from(gameSaves)
        .where(and(
```

```
            eq(gameSaves.userId, input.userId),
            eq(gameSaves.scriptId, input.scriptId)
          ))
          .orderBy(desc(gameSaves.updatedAt)).limit(10);
      }),
  });
```

---

6. 全局状态管理

文件路径： `client/src/contexts/GameContext.tsx`

游戏的核心状态管理，基于 React `useReducer` 实现。所有游戏状态（玩家、故事、NPC、背包、设置等）均集中于此。

6.1 核心类型定义

```
// ── 玩家七维属性 ──
export interface PlayerStats {
  health: number;    // 健康值 0-100
  energy: number;    // 精力值 0-100
  beauty: number;    // 魅力值 0-100
  physique: number;  // 体质值 0-100
  mood: number;      // 心情值 0-100
  stress: number;    // 压力值 0-100
  iq: number;        // 智力值 0-100
}

// ── 玩家完整档案 ──
export interface PlayerProfile {
  name: string;
  gender: "male" | "female" | "other";
  age: number;
  zodiac: string;
  background: string;
  avatarUrl: string;
  location: string;
  weather: string;
  height: number;
  weight: number;
  bust?: number; waist?: number; hips?: number;
  skinTone: string;
  stats: PlayerStats;
  moneyCny: number;
  moneyUsd: number;
  moneyCoins: number;
  traits: string[];
  talents: string[];
  isVirgin: boolean;
  hasFirstKiss: boolean;
  isPregnant: boolean;
  pregnancyDays: number;
}

// ── NPC 档案 ──
export interface NpcProfile {
  id: number;
  name: string;
  gender: string;
  age: number;
```

```
  occupation: string;
  relationship: string;
  meetReason: string;
  background: string;
  avatarUrl: string;
  zodiac: string;
  traits: string[];
  height: number;
  weight: number;
  skinTone: string;
  affection: number;    // 好感度 0-100
  disgust: number;      // 厌恶度 0-100
  heartbeat: number;    // 心动值 0-100
  r18Value: number;     // R18 值 0-100
  unlockedInfo: string[];
  isActive: boolean;
  wealth: number;
}

// ── 故事消息 ──
export interface StoryMessage {
  id: string;
  role: "narrator" | "player" | "system";
  content: string;
  timestamp: Date;
}

// ── AI API 配置 ──
export interface ApiConfig {
  endpoint: string;
  apiKey: string;
  model: string;
  temperature: number;
  maxTokens: number;
  r18Enabled: boolean;
  extraPrompt: string;
  useBuiltinProxy: boolean; // true = 使用内置代理，无需 API Key
}

// ── 外观配置 ──
export interface AppearanceConfig {
  theme: string;      // "sakura-girl" | "orange-soda" | "noble-black-gold"
  font: string;       // 字体标识
  language: string;   // "zh-TW" | "zh-CN" | "en" | "ja" | "ko"
  density: string;    // "comfortable" | "compact"
}
```

```typescript
// ── 流程步骤 ──
export type OnboardingStep =
  | "welcome"
  | "login"
  | "api-config"
  | "appearance"
  | "char-setup"
  | "main";

// ── 游戏页面 ──
export type GamePage = "story" | "social" | "inventory" | "notes" |
"settings" | "scripts";
```

6.2 Reducer 关键 Action 处理

```
// UPDATE_PLAYER_STATS：更新属性值，自动限制在 0-100 范围内
case "UPDATE_PLAYER_STATS": {
  if (!state.player) return state;
  const changes = action.payload;
  const newStats = { ...state.player.stats };
  for (const [key, delta] of Object.entries(changes)) {
    if (key in newStats) {
      const k = key as keyof PlayerStats;
      newStats[k] = Math.max(0, Math.min(100, newStats[k] + (delta as
number)));
    }
  }
  return { ...state, player: { ...state.player, stats: newStats } };
}

// LOAD_SAVE：从存档恢复完整游戏状态，直接跳至 main 步骤
case "LOAD_SAVE": {
  const p = action.payload;
  return {
    ...state,
    saveId: p.saveId,
    player: p.player,
    gameDate: p.gameDate,
    round: p.round,
    storyHistory: p.storyHistory,
    npcs: p.npcs,
    chatMessages: p.chatMessages,
    inventory: p.inventory,
    notes: p.notes,
    scriptData: p.scriptData,
    onboardingStep: "main",  // 直接进入游戏
    currentPage: "story",
  };
}

// SET_SCRIPT_DATA：切换剧本时注入新的世界观设定
case "SET_SCRIPT_DATA":
  return { ...state, scriptData: action.payload };

// RESET_STORY：切换剧本时重置故事历史
case "RESET_STORY":
  return { ...state, storyHistory: [], round: 1 };
```

6.3 Context Provider

```
export function GameProvider({ children }: { children: React.ReactNode }) {
  const [state, dispatch] = useReducer(gameReducer, initialState);

  // 便捷方法：添加故事消息（自动生成唯一 ID）
  const addStory = useCallback((role: StoryMessage["role"], content: string)
=> {
    dispatch({
      type: "ADD_STORY",
      payload: {
        id: `${Date.now()}-${Math.random()}`,
        role,
        content,
        timestamp: new Date()
      },
    });
  }, []);

  // 便捷方法：格式化游戏内时间（如「2024年3月15日 周五 下午」）
  const formatGameDate = useCallback(() => {
    const { year, month, day, hour, dayOfWeek } = state.gameDate;
    const period = hour < 6 ? "深夜" : hour < 12 ? "上午" : hour < 18 ? "下午"
: "晚上";
    return `${year}年${month}月${day}日 周${dayOfWeek} ${period}`;
  }, [state.gameDate]);

  return (
    <GameContext.Provider value={{
      state, dispatch,
      openModal, closeModal, setPage,
      addStory, setAiThinking, formatGameDate, goToStep
    }}>
      {children}
    </GameContext.Provider>
  );
}

// 使用 Hook（在 GameProvider 外部调用会抛出错误）
export function useGame() {
  const ctx = useContext(GameContext);
  if (!ctx) throw new Error("useGame must be used within GameProvider");
  return ctx;
}
```

7. AI 引擎

文件路径： `client/src/lib/aiEngine.ts`

封装所有 AI 调用逻辑，包括开场生成、剧情推进、NPC 对话、API 测试。

7.1 核心类型

```
// AI 剧情推进结果
export interface AiStoryResult {
  sceneTitle?: string;
  narrative: string;
  choices: AiChoice[];
  statChanges?: Partial<PlayerStats & { moneyCny: number }>;
  timeAdvance?: number;  // 推进的小时数
  mood?: string;
  event?: string | null;
  newNpc?: NpcSeed | null;
}

// 选项类型
export interface AiChoice {
  id: string;
  text: string;
  hint?: string;
  statHint?: string;  // 属性变化提示（如「+心情」）
  type?: "action" | "dialogue" | "explore" | "rest" | "special";
}

// 剧本配置（传入 AI 的世界观信息）
export interface ScriptConfig {
  name: string;
  genre: string;
  description?: string;
  worldSetting: string;  // JSON 字符串
}
```

7.2 双模式 LLM 调用

```
/**
 * 通过后端代理调用 LLM（无需 API Key）
 * 调用路径：前端 → /api/trpc/game.aiProxy → 服务端 invokeLLM
 */
async function callLLMViaBuiltinProxy(
  messages: Array<{ role: "system" | "user" | "assistant"; content: string
}>,
  temperature = 0.85,
  maxTokens = 800
): Promise<string> {
  const payload = { messages, temperature, maxTokens };
  const res = await fetch("/api/trpc/game.aiProxy", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ "0": { json: payload } }),
  });
  if (!res.ok) throw new Error(`Proxy error: ${res.status}`);
  const data = await res.json() as {
    result?: { data?: { json?: { success: boolean; content: string; error?:
string } } }
  };
  const result = data?.result?.data?.json;
  if (!result?.success) throw new Error(result?.error || "AI proxy failed");
  return result.content;
}

/**
 * 直接调用外部 OpenAI 兼容 API（用户自定义 Key）
 */
async function callLLM(
  apiConfig: ApiConfig,
  messages: Array<{ role: "system" | "user" | "assistant"; content: string
}>,
  temperature?: number,
  maxTokens?: number
): Promise<string> {
  // 优先使用内置代理
  if (apiConfig.useBuiltinProxy || !apiConfig.apiKey) {
    return callLLMViaBuiltinProxy(messages, temperature, maxTokens);
  }
  // 使用用户配置的外部 API
  const endpoint = apiConfig.endpoint.replace(/\/$/, "");
  const url = endpoint.endsWith("/chat/completions")
```

```
    ? endpoint : `${endpoint}/chat/completions`;
  const res = await fetch(url, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${apiConfig.apiKey}`,
    },
    body: JSON.stringify({
      model: apiConfig.model || "gpt-4o-mini",
      messages,
      temperature: temperature ?? apiConfig.temperature ?? 0.85,
      max_tokens: maxTokens ?? apiConfig.maxTokens ?? 800,
      response_format: { type: "json_object" },
    }),
  });
  if (!res.ok) {
    const err = await res.text();
    throw new Error(`API error (${res.status}): ${err.slice(0, 200)}`);
  }
  const data = await res.json();
  return data.choices?.[0]?.message?.content ?? "";
}
```

7.3 开场生成函数

```
/**
 * 生成游戏开场（玩家点击「开始游戏」时调用）
 * 返回：200-300字开场叙事 + 初始三选一 + 1-3个初始 NPC
 */
export async function generateGameOpening(
  player: PlayerProfile,
  apiConfig: ApiConfig,
  lang: string,
  scriptConfig?: ScriptConfig
): Promise<AiInitResult> {
  const systemPrompt = buildSystemPrompt(player, apiConfig, lang,
scriptConfig);

  // 如果有剧本设定，在 userPrompt 中注入世界观
  const scriptContext = scriptConfig
    ? `【剧本世界观】${scriptConfig.name}（${scriptConfig.genre}）\n`
    : "";

  const userPrompt = `${scriptContext}请为玩家「${player.name}」生成游戏开场。
要求：
- 开场叙事 200-300 字，第二人称，沉浸感强，有画面感
- 自然引入 1-3 个 NPC（符合世界观）
- 生成 3 个差异化的初始选项

输出 JSON：
{
  "opening": "开场叙事文字",
  "sceneTitle": "场景标题（4-8字）",
  "choices": [
    {"id": "c1", "text": "选项1", "hint": "影响暗示", "type": "action"},
    {"id": "c2", "text": "选项2", "hint": "影响暗示", "type": "explore"},
    {"id": "c3", "text": "选项3", "hint": "影响暗示", "type": "dialogue"}
  ],
  "initialNpcs": [
    {
      "name": "NPC名", "gender": "female", "age": 25,
      "occupation": "职业", "relationship": "关系",
      "meetReason": "相遇原因", "zodiac": "星座",
      "traits": ["性格1", "性格2"]
    }
  ]
}`;
```

```
  const raw = await callLLM(apiConfig, [
    { role: "system", content: systemPrompt },
    { role: "user", content: userPrompt },
  ]);

  try {
    const cleaned = raw.replace(/```json\n?/g, "").replace(/```\n?/g,
"").trim();
    const result = JSON.parse(cleaned);
    return {
      openingNarrative: result.opening || result.narrative || raw.slice(0,
300),
      firstChoices: result.choices || getDefaultChoices(),
      initialNpcs: result.initialNpcs || [],
      sceneTitle: result.sceneTitle,
    };
  } catch {
    return {
      openingNarrative: raw.slice(0, 300) || "故事就此开始...",
      firstChoices: getDefaultChoices(),
      initialNpcs: [],
    };
  }
}
```

7.4 剧情推进函数

```
/**
 * 推进故事（玩家每次做出选择后调用）
 * 包含 AI 推动机制：
 * - 回合数 % 4 == 0：强制意外转折
 * - 回合数 % 7 == 0：引入新 NPC
 */
export async function advanceStory(
  player: PlayerProfile,
  choice: string,
  recentHistory: string,
  apiConfig: ApiConfig,
  lang: string,
  scriptConfig?: ScriptConfig,
  round?: number
): Promise<AiStoryResult> {
  const systemPrompt = buildSystemPrompt(player, apiConfig, lang,
scriptConfig);
  const roundNum = round || 1;

  // 根据回合数注入特殊事件提示
  const eventHint = roundNum % 4 === 0
    ? "\n⚡ 本回合是关键节点，请引入一个意外转折（偶遇重要人物/发现秘密/突发危机/意外机
遇）。"
    : roundNum % 7 === 0
    ? "\n✨ 本回合可以引入一个新 NPC 角色，为故事增添新的人际关系维度。"
    : "";

  const userPrompt = `## 最近的故事记录
${recentHistory}

## 玩家的选择（第${roundNum}回合）
「${choice}」
${eventHint}

请根据玩家的选择，继续推进故事。要求：
- 叙事要与上一段自然衔接，展现选择带来的直接后果
- 描写场景变化、人物反应、情绪起伏（150-250字）
- 结尾留下悬念或期待感

输出 JSON（choices 必须恰好3个，类型各不相同）：
{
  "sceneTitle": "场景标题（可选）",
  "narrative": "故事叙事",
```

```
  "choices": [
    {"id": "c1", "text": "行动选项", "hint": "影响", "statHint": "+心情",
"type": "action"},
    {"id": "c2", "text": "对话选项", "hint": "影响", "statHint": "-精力",
"type": "dialogue"},
    {"id": "c3", "text": "休息选项", "hint": "影响", "statHint": "+精力",
"type": "rest"}
  ],
  "statChanges":
{"health":0,"energy":-5,"mood":10,"stress":-5,"beauty":0,"physique":0,"iq":0,"
  "timeAdvance": 2,
  "mood": "场景氛围关键词",
  "event": null,
  "newNpc": null
}`;

  const raw = await callLLM(apiConfig, [
    { role: "system", content: systemPrompt },
    { role: "user", content: userPrompt },
  ]);

  try {
    const cleaned = raw.replace(/```json\n?/g, "").replace(/```\n?/g,
"").trim();
    const result = JSON.parse(cleaned) as AiStoryResult;

    // 确保 choices 恰好3个
    if (!result.choices || result.choices.length === 0) {
      result.choices = getDefaultChoices();
    } else if (result.choices.length > 3) {
      result.choices = result.choices.slice(0, 3);
    } else while (result.choices.length < 3) {
      result.choices.push(getDefaultChoices()[result.choices.length]);
    }
    return result;
  } catch {
    return {
      narrative: raw.slice(0, 300) || "故事继续发展...",
      choices: getDefaultChoices(),
      statChanges: { energy: -3, mood: 2 },
      timeAdvance: 1,
      mood: "平静",
    };
  }
}
```

```
// 默认选项（AI 失败时的兜底）
function getDefaultChoices(): AiChoice[] {
  return [
    { id: "c1", text: "主动出击，把握机会", hint: "可能有意外收获", type:
"action" },
    { id: "c2", text: "静观其变，等待时机", hint: "保存实力", type: "rest" },
    { id: "c3", text: "寻找线索，深入了解", hint: "增长见识", type: "explore" },
  ];
}
```

---

8. 云端存档 Hook

文件路径： `client/src/hooks/useSaveGame.ts`

封装云端存档的所有逻辑，提供 `saveGame` 和 `restoreFromSaveData` 两个核心方法。

```
import { useCallback, useRef } from "react";
import { trpc } from "@/lib/trpc";
import { useGame } from "@/contexts/GameContext";
import type {
  PlayerProfile, NpcProfile, StoryMessage,
  ChatMessage, InventoryItem, Note, GameDate
} from "@/contexts/GameContext";

export function useSaveGame() {
  const { state, dispatch } = useGame();
  const saveGameMutation = trpc.game.saveGame.useMutation();
  const isSavingRef = useRef(false); // 防重入锁，避免并发存档

  /**
   * 将当前游戏状态序列化并保存到云端
   * @param slotName 存档槽名称，默认 "auto"
   */
  const saveGame = useCallback(async (slotName = "auto") => {
    if (isSavingRef.current) return null; // 防止并发
    isSavingRef.current = true;
    try {
      const { player, npcs, storyHistory, chatMessages, inventory,
              notes, gameDate, round, scriptData } = state;
      if (!player) return null;

      const result = await saveGameMutation.mutateAsync({
        slotName,
        playerData: JSON.stringify(player),
        npcsData: JSON.stringify(npcs),
        storyData: JSON.stringify(storyHistory),
        chatData: JSON.stringify(chatMessages),
        inventoryData: JSON.stringify(inventory),
        notesData: JSON.stringify(notes),
        scriptData: JSON.stringify(scriptData),
        round,
        gameYear: gameDate.year,
        gameMonth: gameDate.month,
        gameDay: gameDate.day,
      });

      if (result.saveId) {
        dispatch({ type: "SET_SAVE_ID", payload: result.saveId });
      }
      return result;
    } catch (e) {
```

```
    console.error("[useSaveGame] saveGame error:", e);
    return null;
  } finally {
    isSavingRef.current = false;
  }
}, [state, saveGameMutation, dispatch]);

/**
 * 从云端存档数据恢复游戏状态
 * 注意：storyHistory 中的 timestamp 需要从 string 转回 Date
 */
const restoreFromSaveData = useCallback((saveData: {
  id: number;
  playerData: string | null;
  npcsData: string | null;
  storyData: string | null;
  chatData: string | null;
  inventoryData: string | null;
  notesData: string | null;
  scriptData: string | null;
  round: number | null;
  gameYear: number | null;
  gameMonth: number | null;
  gameDay: number | null;
}) => {
  try {
    const player: PlayerProfile = saveData.playerData
      ? JSON.parse(saveData.playerData) : null;
    if (!player) return false;

    const npcs: NpcProfile[] = saveData.npcsData
      ? JSON.parse(saveData.npcsData) : [];

    // 反序列化时恢复 Date 类型
    const storyHistory: StoryMessage[] = saveData.storyData
      ? JSON.parse(saveData.storyData).map((m: StoryMessage) => ({
          ...m,
          timestamp: new Date(m.timestamp),
        }))
      : [];

    const chatMessages: Record<number, ChatMessage[]> = saveData.chatData
      ? JSON.parse(saveData.chatData) : {};
    const inventory: InventoryItem[] = saveData.inventoryData
      ? JSON.parse(saveData.inventoryData) : [];
    const notes: Note[] = saveData.notesData
```

```
        ? JSON.parse(saveData.notesData) : [];
      const scriptData: Record<string, unknown> = saveData.scriptData
        ? JSON.parse(saveData.scriptData) : {};

      const gameDate: GameDate = {
        year: saveData.gameYear ?? 2000,
        month: saveData.gameMonth ?? 1,
        day: saveData.gameDay ?? 1,
        hour: 8,
        dayOfWeek: "一",
      };

      dispatch({
        type: "LOAD_SAVE",
        payload: {
          saveId: saveData.id, player, gameDate,
          round: saveData.round ?? 1, storyHistory,
          npcs, chatMessages, inventory, notes, scriptData,
        },
      });
      return true;
    } catch (e) {
      console.error("[useSaveGame] restoreFromSaveData error:", e);
      return false;
    }
  }, [dispatch]);

  return {
    saveGame,
    restoreFromSaveData,
    isSaving: saveGameMutation.isPending,
  };
}
```

---

9. 单元测试

文件路径： `server/scripts.test.ts`

使用 Vitest 对剧本路由进行单元测试，无需真实数据库连接（测试路由结构和静态数据）。

```typescript
/**
 * scripts.test.ts — 剧本路由单元测试
 * 测试覆盖：模板数据完整性、类型标签、路由结构
 */
import { describe, expect, it } from "vitest";
import { appRouter } from "./routers";
import type { TrpcContext } from "./_core/context";

// 模拟公开 Context（无需真实 DB）
function createPublicContext(): TrpcContext {
  return {
    user: null,
    req: {} as TrpcContext["req"],
    res: {} as TrpcContext["res"],
  };
}

describe("scripts router", () => {
  it("getTemplates should return an array of templates", async () => {
    const caller = appRouter.createCaller(createPublicContext());
    const templates = await caller.scripts.getTemplates();
    expect(Array.isArray(templates)).toBe(true);
    expect(templates.length).toBeGreaterThan(0);
  });

  it("each template should have required fields", async () => {
    const caller = appRouter.createCaller(createPublicContext());
    const templates = await caller.scripts.getTemplates();
    for (const tpl of templates) {
      expect(tpl).toHaveProperty("name");
      expect(tpl).toHaveProperty("genre");
      expect(tpl).toHaveProperty("description");
      expect(tpl).toHaveProperty("worldSetting");
      expect(tpl).toHaveProperty("coverColor");
      expect(tpl).toHaveProperty("coverIcon");
      expect(typeof tpl.name).toBe("string");
      expect(tpl.name.length).toBeGreaterThan(0);
    }
  });

  it("getGenres should return an array of genre strings", async () => {
    const caller = appRouter.createCaller(createPublicContext());
    const genres = await caller.scripts.getGenres();
    expect(Array.isArray(genres)).toBe(true);
    expect(genres.length).toBeGreaterThan(0);
```

```
    expect(genres).toContain("现代都市");
    expect(genres).toContain("古风仙侠");
  });

  it("template worldSetting should be valid JSON with required fields",
async () => {
    const caller = appRouter.createCaller(createPublicContext());
    const templates = await caller.scripts.getTemplates();
    for (const tpl of templates) {
      expect(() => JSON.parse(tpl.worldSetting)).not.toThrow();
      const ws = JSON.parse(tpl.worldSetting);
      expect(ws).toHaveProperty("era");
      expect(ws).toHaveProperty("theme");
    }
  });
});

describe("game router - aiProxy structure", () => {
  it("appRouter should have game.aiProxy mutation", () => {
    const caller = appRouter.createCaller(createPublicContext());
    expect(typeof caller.game.aiProxy).toBe("function");
  });

  it("appRouter should have all scripts procedures", () => {
    const caller = appRouter.createCaller(createPublicContext());
    expect(typeof caller.scripts.getTemplates).toBe("function");
    expect(typeof caller.scripts.listScripts).toBe("function");
    expect(typeof caller.scripts.createScript).toBe("function");
    expect(typeof caller.scripts.activateScript).toBe("function");
    expect(typeof caller.scripts.deleteScript).toBe("function");
    expect(typeof caller.scripts.getGenres).toBe("function");
  });
});
```